

Paper Number: P1068R3
Title: Vector API for random number generation
Authors: Ilya Burylov <ilya.burylov@intel.com>
Pavel Dyakov <pavel.dyakov@intel.com>
Ruslan Arutyunyan <ruslan.arutyunyan@intel.com>
Andrey Nikolaev <andrey.nikolaev@intel.com>

Audience: LEWG
Date: 2020-01-13

I. Introduction

C++11 introduced a comprehensive mechanism to manage generation of random numbers in the `<random>` header file.

We propose to introduce an additional API based on iterators in alignment with algorithms definition. `simd`-type based interface presented in previous paper revisions will be submitted as a separate paper.

II. Revision history

Key changes for R3 compared with R2:

- Removed execution policies from API, based on Cologne meeting decision.
- Removed `simd`-based API, for separate consideration as a follow up paper, based on corresponding TS results.
- Added formal wording section for iterators-based API.

Key changes for R2 compared with R1:

- Proposed API for switching between Sequentially consistent and Sequentially inconsistent vectorized results.
- Added performance data measured on the prototype to show price for sequentially consistent results.
- Extended description of the role of `generate_canonical` in distributions implementations.
- Reworked *Possible approaches to address the problem* chapter to focus on two main approaches under consideration.

Key changes for R1 compared with R0:

- Extended the list of possible approaches with `simd` type direct usage.
- Added performance data measured on the prototype.
- Changed the recommendation to a combined approach.

III. Motivation and Scope

The C++11 random-number API is essentially a scalar one. Stateful nature and the scalar definition of underlying algorithms prevent auto-vectorization by compiler.

However, most existing algorithms for generation of pseudo- [and quasi-]random sequences allow algorithmic rework to generate numbers in batches, which allows the implementation to utilize `simd`-based HW instruction sets.

Internal measurements show significant scaling over `simd`-size for key baseline Engines yielding a substantial performance difference on the table on modern HW architectures.

Extension and/or modification of the list of supported Engines and/or Distributions is out of the scope of this proposal.

IV. Libraries and other languages

Vector APIs are common for the area of generation random numbers. Examples:

- * Intel® Math Kernel Library (Intel® MKL)

- Statistical Functions component includes Random Number Generators C vector based API

- * Java* java.util.Random

- Has doubles(), ints(), longs() methods to provide a stream of random numbers

- * Python* NumPy* library

- NumPy array has a method to be filled with random numbers

- * NVIDIA* cuRAND

- host API is vector based

Intel MKL can be an example of the existing vectorized implementation for verity of engines and distributions. Existing API is C [1] (and FORTRAN), but the key property which allows enabling vectorization is vector-based interface.

Another example of implementation can be intrinsics for the Short Vector Random Number Generator Library [2], which provides an API on simd level and can be considered an example of internal implementation for proposed modifications.

V. Problem description

Main flow of random number generation is defined as a 3-level flow.

User creates Engine and Distribution and calls operator() of Distribution object, providing Engine as a parameter:

```
std::vector<float> v(10);

std::mt19937 gen(777);
std::uniform_real_distribution<> dis(1.0f, 2.0f);

for(auto& e1 : v)
{
    e1 = dis(gen);
}
```

operator() of a Distribution typically (but not necessarily so) implements scalar algorithm and calls generate_canonical(), passing Engine object further down:

```
uniform_real_distribution::operator()(_URNG& __gen)
{
    return (b() - a()) * generate_canonical<_RealType>(__gen) + a();
}
```

It is necessary to note, that C++ standard does not require calling generate_canonical() function inside any distribution implementation and it does not specify the number of Engine numbers per distribution number. Having said that, 3 main standard library implementations share the same schema, described here.

generate_canonical() has a main intention to generate enough entropy for the type used by Distribution, and it calls operator() of an Engine one or more times (number of times is a compile-time constant):

```
_RealType generate_canonical(_URNG& __gen())
{
    ...
    _RealType _Sp = __gen() - _URNG::min();
    for (size_t __i = 1; __i < __k; ++__i, __base *= _Rp)
        _Sp += (__gen() - _URNG::min()) * __base;
    return _Sp / _Rp;
}
```

operator() of an Engine is (almost) always stateful, with non-trivial dependencies between iterations, which prevents any auto-vectorization:

```
mersenne_twister_engine<...>::operator() ()
{
    const size_t __j = (__i + 1) % __n;
    ...
    const result_type _Yp = (__x[__i] & ~__mask) | (__x[__j] & __mask);
    const size_t __k = (__i + __m) % __n;
    __x[__i] = __x[__k] ^ __rshift<1>(_Yp) ^ (__a * (_Yp & 1));
    result_type __z = __x[__i] ^ (__rshift<__u>(__x[__i]) & __d);
    __i = __j;
    ...
    return __z ^ __rshift<__l>(__z);
}
```

operator() of the most distributions can be implemented in a way, which compiler can inline and auto-vectorize. generate_canonical() adds additional challenge for the compiler due to loop, but it is resolvable. operator() on the engine level is the key showstopper for the auto-vectorization.

VI. Iterators-based API

The following API extension is targeting to cover generation of bigger chunks of random numbers, which allows internal optimizations hidden inside implementation.

API of Engines and Distributions is extended with iterators based API.

```
std::array<double, arrayLength> stdArray;
std::experimental::minstd_rand0 genStd(555);
std::experimental::uniform_real_distribution<double> disFloat(0.0, 1.0);
disFloat(stdArray.begin(), stdArray.end(), genStd);
```

The output of this function may or may not be equivalent to the scalar calls of the scalar API:

```
for(double& d : arrayLength) {
    d = distFloat(genStd);
}
```

VII. Wording proposal

26.6.2.3 Uniform random bit generator requirements [rand.req.urng]

...

```
template<class G, class ForwardIterator >
concept uniform_random_bit_generator =
    invocable<G&> && unsigned_integral<invoke_result_t<G&>> &&
    requires(G& g, ForwardIterator begin, ForwardIterator end) {
        { G::min() } -> same_as<invoke_result_t<G&>>;
        { G::max() } -> same_as<invoke_result_t<G&>>;
        { g(begin, end) } -> same_as<void>;
    };
```

...

26.6.2.4 Random number engine requirements [rand.req.eng]

...

The template argument for parameters named `ForwardIterator` shall meet the `Cpp17ForwardIterator` requirements (23.3.5.4).

Table 92: Random number engine requirements [tab:rand.req.eng]

Expression	Return type	Pre/post-condition	Complexity
...			
<code>e()</code>	<code>T</code>	Advances <code>e</code> 's state <code>e_i</code> to <code>e_{i+1} = TA(e_i)</code> and returns <code>GA(e_i)</code>	per 26.6.2.3
<code>e(ForwardIterator first, ForwardIterator last)</code>	<code>void</code>	With $N = \text{last} - \text{first}$, assigns the result of evaluations of <code>e()</code> through each iterator in the range <code>[first, first + N)</code> .	$O(N)$
...			

26.6.2.5 Random number engine adaptor requirements [rand.req.adapt]

No changes

26.6.2.6 Random number distribution requirements [rand.req.dist]

The template argument for parameters named `ForwardIterator` shall meet the `Cpp17ForwardIterator` requirements (23.3.5.4).

Table 93: Random number distribution requirements [tab:rand.req.dist]

Expression	Return type	Pre/post-condition	Complexity
...			
<code>d(g)</code>	<code>T</code>	With <code>p = d.param()</code> , the sequence of numbers returned by successive invocations with the same object <code>g</code> is randomly distributed according to the associated <code>p(z {p})</code> or <code>P(z_i {p})</code> function.	amortized constant number of invocations of <code>g</code>
<code>d(g,p)</code>	<code>T</code>	The sequence of numbers returned by successive invocations with the same objects <code>g</code> and <code>p</code> is randomly distributed according to the associated <code>p(z {p})</code> or <code>P(z_i {p})</code> function.	amortized constant number of invocations of <code>g</code>

d(ForwardIterator first, ForwardIterator last, g)	void	With $N = \text{last} - \text{first}$ and $p = d.\text{param}()$, the sequence of numbers assigned through each iterator in $[\text{first}, \text{first} + N)$ is randomly distributed according to the associated $p(z \{p\})$ or $P(z_i \{p\})$ function.	$O(N)$
d(ForwardIterator first, ForwardIterator last, g, p)	void	With $N = \text{last} - \text{first}$, the sequence of numbers assigned through each iterator in $[\text{first}, \text{first} + N)$ is randomly distributed according to the associated $p(z \{p\})$ or $P(z_i \{p\})$ function.	$O(N)$
...			

VIII. Design considerations

See other design considerations in P1068R1 and P1068R2.

IX. Performance results

Implementation approaches were prototyped in part of Distribution API (and Engine API, where required for the use case). Short Vector Random Number Generator Library [2] was used as an underlying vectorization engine. LLVM* libc++ 8.0 implementation was used as a baseline implementation. See P1068R2 for performance results.

X. Impact on the standard

This is a library-only extension. It adds new member functions to some classes. This change is ABI compatible with existing random numbers generation functionality.

XI. References

1. Intel MKL documentation:
<https://software.intel.com/en-us/mkl-developer-reference-c-2019-beta-basic-generators>
2. Intrinsics for the Short Vector Random Number Generator Library
<https://software.intel.com/en-us/node/694866>
3. Box-Muller method
https://en.wikipedia.org/wiki/Box%E2%80%93Muller_transform
4. Inverse transform sampling
https://en.wikipedia.org/wiki/Inverse_transform_sampling

Legal Disclaimer & Optimization Notice

INFORMATION IN THIS DOCUMENT IS PROVIDED "AS IS". NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. INTEL ASSUMES NO LIABILITY WHATSOEVER AND INTEL DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO THIS INFORMATION INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

Software and workloads used in performance tests may have been optimized for performance only on Intel microprocessors. Performance tests, such as SYSmark and MobileMark, are measured using specific computer systems, components, software, operations and functions. Any change to any of those factors may cause the results to vary. You should consult other information and performance tests to assist you in fully evaluating your contemplated purchases, including the performance of that product when combined with other products.

Copyright © 2019, Intel Corporation. All rights reserved. Intel, Pentium, Xeon, Xeon Phi, Core, VTune, Cilk, and the Intel logo are trademarks of Intel Corporation in the U.S. and other countries.

Optimization Notice

Intel's compilers may or may not optimize to the same degree for non-Intel microprocessors for optimizations that are not unique to Intel microprocessors. These optimizations include SSE2, SSE3, and SSSE3 instruction sets and other optimizations. Intel does not guarantee the availability, functionality, or effectiveness of any optimization on microprocessors not manufactured by Intel. Microprocessor-dependent optimizations in this product are intended for use with Intel microprocessors. Certain optimizations not specific to Intel microarchitecture are reserved for Intel microprocessors. Please refer to the applicable product User and Reference Guides for more information regarding the specific instruction sets covered by this notice.

Notice revision #20110804